

Guarding `getQuantSplines` against unidentifiable RT-normalization fits

2026-08-11

Contents

Summary	1
How it surfaced	2
What the data actually looks like	2
Why the fit is ill-posed	3
The failure that matters	3
Proposed guard	4
Identifiability check	4
In <code>getQuantSplines</code>	4
Required downstream change	4
Alternatives considered	5
Verification	5
What this does not address	5

Summary

`getQuantSplines` fits a 10-coefficient cubic spline per MS file, mapping observed iRT to median log2 abundance, and uses it to normalize quantification across runs. It applies no check that the file actually contains enough distinct retention times to identify such a spline.

On files that do not, two things happen. Occasionally the least-squares solve throws `LinearAlgebra.SingularException` and kills the search. Far more often it returns silently, having fitted 10 parameters through a rank-5 system, and the “normalization” derived from it is noise applied to every precursor in that file.

The crash is how this was found. The silent case is the reason to fix it.

Recommendation: decline to fit when the spline is not identifiable, skip normalization for that file, and say so in the log. Do not force a fit.

How it surfaced

A local `create_app` build of `feat/integ-minwidth5-order3-portable` failed in the `SearchDIA_yeast` snoop target:

```
LinearAlgebra.SingularException(10)
  MaxLFQSearch.jl:195  summarize_results!
  normalizeQuant.jl:137  normalizeQuant
  normalizeQuant.jl:55   getQuantSplines
  uniformBasisCubicSpline.jl:227  UniformSpline
```

Three tests isolated the trigger:

test	result
branch as-is	fails, <code>SingularException(10)</code>
3rd-order WH smoothing disabled (<code>WH_ORDER3_LAMBDA[] = 0</code>)	still fails, <code>SingularException(2)</code>
feature/portable-packaging-phase1 (no integration changes)	passes
branch with only <code>apex_padded ±2</code> reverted to <code>±1</code>	passes

So the trigger is the minimum 5-scan integration width — a two-line change to the boundary search in `getIntegrationBounds!`.

It is worth being precise about what that means: the integration change did not introduce a defect in `normalizeQuant`. It perturbed quantification by a handful of PSMs on one file, and that was enough to tip an already-unidentifiable fit from “silently wrong” to “throws”. Any change anywhere upstream of quantification can do the same.

What the data actually looks like

Instrumenting the fit to report its inputs, on the three-file yeast fixture:

With the minimum-width change (fails):

file	PSMs	N_sample	bin_size	distinct iRTs	median iRT span
lowsignal.arrow	14	14	1	13	0.851
rep1.arrow	8,705	100	87	100	20.536
rep2.arrow	8,710	100	87	100	20.297

With it reverted (passes):

file	PSMs	N_sample	bin_size	distinct iRTs	median iRT span
lowsignal.arrow	18	18	1	17	1.134
rep1.arrow	8,634	100	86	100	19.448
rep2.arrow	8,636	100	86	100	19.827

In every case the spline has `n_coeffs = n_knots + 3 = 10` coefficients.

Two observations:

1. **The margin is four PSMs.** 18 passes, 14 throws. Nothing about the current code makes 18 safe — it is simply on the tolerable side of a cliff.
2. **bin_size = 1.** The binning has collapsed entirely. `median_rts` is not a set of bin medians; it is just the sorted per-PSM iRTs. The averaging that makes this estimator sensible on `rep1/rep2` (87 PSMs per bin) is absent.

`lowsignal` is in the fixture deliberately — `snoop.jl` describes it as “1 MB of void volume that yields zero PSMs... the only file that compiles the fallback paths.” It is a stand-in for a real user’s failed run, and real users have those.

Why the fit is ill-posed

`_setup_knots` places `n_knots = 7` knots uniformly across `[min(t), max(t)]`, and `_build_numeric_design_matrix` gives each row four non-zero entries in the columns of the cubic B-splines covering that point. A column is all-zero whenever no data point falls in its support.

Measured directly on synthetic distributions matching the failing file’s shape (14 points, span 0.851, `n_knots = 7`, so `X` is 14x10):

iRT distribution	rank of <code>X</code>	all-zero columns
evenly spaced, 14 points	9	1
12 clustered + 2 outliers	6	2
13 clustered + 1 outlier	5	3
two tight clumps of 7	8	2

Every one of these is rank-deficient, including the evenly-spaced case. Ten basis functions cannot be identified from fourteen points spanning 0.85 iRT units, however they are arranged. The system is underdetermined before any numerical question arises.

Whether that surfaces as an exception depends on the shape of `X`. With more rows than columns, `\` takes a QR path that returns a solution for a rank-deficient system without complaining — which is why none of the four synthetic cases above throws. The exception requires an additional condition that this analysis did not isolate; the observed stack goes through `lu`, which Julia selects for square systems, implying a file where `N_sample` lands exactly on `n_coeffs = 10`.

That uncertainty does not affect the recommendation, and is itself the point. The crash is a narrow special case of a broad problem. Fixing only the crash would leave the common case — a rank-5 fit accepted silently — untouched.

The failure that matters

When `\` does not throw, `getQuantSplines` returns a spline fitted through a rank-5-of-10 system. That spline is then used, in `getQuantCorrections`, on equal footing with the splines from `rep1` and `rep2`:

```
for (key, spline) in pairs(quant_splines)
    median_quant[j, i] = spline(rt_grid[i])    # every file contributes equally
    j += 1
end
median_quant = reshape(median(median_quant, dims = 1), (N,))
```

The cross-file median that defines the normalization target is computed with the degenerate file weighted the same as files with 8,700 PSMs. Its spline is then also evaluated *outside* the 0.85-unit range it was fitted on — `rt_grid` spans the union of all files' RT ranges, roughly 20 units — where `UniformSpline` clamps to the endpoints. The correction it contributes is an extrapolated constant derived from noise.

Then in `applyNormalization!`, every precursor in that file is scaled by it:

```
norm_vals[i] = 2^(log2(max(psms[i, quant_col], 0.0)) - hc)
```

So the cost of the missing guard is not primarily a crashed search. It is that low-signal files silently receive, and silently contribute, a meaningless RT-dependent abundance correction.

Proposed guard

Identifiability check

```
"""
    _quant_spline_identifiable(median_rts, n_coeffs) -> Bool

Whether a cubic spline with `n_coeffs` coefficients can be identified from these
bin centres. Requires at least `n_coeffs` *distinct* retention times and a
non-degenerate span; fewer distinct values than coefficients leaves at least one
B-spline basis column with no support, so the least-squares system is rank
deficient regardless of how the points are arranged.
"""

function _quant_spline_identifiable(median_rts::AbstractVector, n_coeffs::Int)
    length(median_rts) >= n_coeffs || return false
    length(unique(median_rts)) >= n_coeffs || return false
    return (maximum(median_rts) - minimum(median_rts)) > 0
end
```

In `getQuantSplines`

```
n_coeffs = spline_n_knots + 3
if !_quant_spline_identifiable(median_rts, n_coeffs)
    @user_warn "Skipping quant normalization for $(basename(fpath)): " *
        "$(length(unique(median_rts))) distinct RT bins over a span of " *
        "$(round(maximum(median_rts) - minimum(median_rts), digits=3)) " *
        "cannot identify a $(n_coeffs)-coefficient spline. This file's " *
        "abundances are left uncorrected and excluded from the cross-file median."
    continue # no entry inserted for this file
end
splinefit = UniformSpline(median_quant, median_rts, 3, spline_n_knots)
insert!(quant_splines, fpath, splinefit)
```

Skipping rather than fitting is deliberate. A file that cannot support the fit also cannot contribute usefully to a cross-file median; including a forced fit adds noise to the target that every *other* file is then corrected against.

Required downstream change

`getQuantCorrections` iterates `pairs(quant_splines)` and needs no change — a skipped file simply does not participate, which is the desired behaviour.

`applyNormalization!` does need one:

```
correction_spline = corrections[fpath] # KeyError for a skipped file
becomes
correction_spline = get(corrections, fpath, nothing)
if correction_spline === nothing
    # No spline for this file: copy the raw column through unchanged so the
    # `_normalized` column exists with the same schema everywhere downstream.
    psms[!, norm_quant_col] = Float64.(psms[!, quant_col])
    writeArrow(fpath, psms)
    continue
end
```

Emitting the `_normalized` column unchanged, rather than omitting it, keeps the Arrow schema uniform across files — downstream readers assume every file has it.

Alternatives considered

Fit with a penalty instead of skipping. `UniformSplinePenalized` already exists and `_penalized_spline_system` supports a ridge term, so $H = X'X + \lambda D$ is positive-definite and cannot be singular. This removes the crash but keeps the silent problem: it still produces a correction for a file with no information to justify one, and still lets that file into the cross-file median.

Lower `spline_n_knots` adaptively for small files. Reduces `n_coeffs` until identifiable. More faithful in principle, but it changes the estimator per file, so files are then normalized against a median assembled from splines of differing flexibility. Harder to reason about than declining.

Deduplicate identical median RTs. Addresses only one of several ways the matrix loses rank (the failing file has 13 distinct values out of 14, so deduplication would remove exactly one point and change nothing).

Verification

1. `SearchDIA_yeast` passes with the minimum-width change in place — the case that currently fails.
2. `rep1/rep2` splines are byte-identical to today's, confirming healthy files are untouched. This matters: the guard must not perturb existing results.
3. A unit test over the degenerate shapes in the rank table above, asserting the guard declines and no exception escapes.
4. The *ecoli* searches (which pass today, both *Altimeter* and *Prosit*) produce unchanged precursor and protein-group counts.

Point 2 is the one to be strict about. The purpose here is to stop normalizing files that cannot support it, not to change normalization for files that can.

What this does not address

- **Why `bin_size` collapses to 1.** `N_sample = min(N, nprec)` means a file with 14 PSMs gets 14 bins of one PSM each. The guard rejects the resulting fit, but the binning logic is arguably wrong well before that point — it should probably require a minimum bin occupancy rather than a minimum bin count.

- **Whether the minimum 5-scan integration width is correct.** It is the trigger, not the defect, and it reduced `lowsignal` from 18 identified PSMs to 14. That drop is worth understanding separately: widening the integration window should not usually cost identifications.
- **The `SingularException` special case.** The guard prevents it by rejecting the fits that produce it, but the precise condition under which `\` throws rather than silently returning a rank-deficient solution was not pinned down.